52% of LLM-Generated Z3 Proofs Are Invalid: Why Theorem Proving Benchmarks Need Proof Certificates

Vishnu Kchitti
vishnu@arbiter.dev

April 2026

Abstract

LLM-based theorem provers now report near-perfect scores on miniF2F, yet these headline numbers conflate "the solver returned `sat`/`unsat`" with "the proof is correct." We audited 23 LLM-generated Z3 proof modules targeting the 35-problem miniF2F-hard tier (AIME and IMO problems) and found that **only 3 (13%) are fully valid**, 8 (35%) are partial, and **12 (52%) are outright invalid**—producing a trust gap between a reported 65.7% solve rate and an actual 8.6% certified rate. We taxonomize seven failure modes including vacuous encoding, assumed derivations, silent crashes, encoding mismatch, trivial arithmetic tails, missing quantifiers, and partial proofs. To close this gap, we introduce a certification pipeline built on Knuckledragger (a Z3 proof assistant with tamper-proof `Proof` objects) and SymPy (for algebraic pre-computation), achieving **89.3% certified on the full 244-problem miniF2F** with Claude Sonnet 4.5 and 70.8% ± 3.1% certified with gpt-5.4-mini across three seeds. We argue that *all* SMT-based benchmarks should report certificate rates alongside solve rates, and release our audit corpus and certification pipeline to facilitate this.

1 Introduction

The miniF2F benchmark [1] has become the de facto leaderboard for LLM-based formal theorem proving. Recent systems report extraordinary numbers: Seed-Prover achieves ∼100% on the test split [2], HILBERT reaches 99.2% [3], Kimina-Prover reports 92.2% [4], and Goedel-Prover-V2 achieves 90.4% [5]. These results are rightly celebrated—they demonstrate that large language models can generate non-trivial formal proofs.

But what exactly is being measured? In Lean-based systems, the Lean kernel type-checks every proof term, providing a genuine certificate of correctness. In SMT-based approaches, however, the situation is murkier. When a Z3 script returns `unsat`, it means the solver found the negated formula unsatisfiable—but it says nothing about whether the *encoding* faithfully represents the original mathematical claim. An LLM can produce a Z3 script that "solves" a problem by asserting the conclusion as a premise, by encoding a weaker statement, or by exploiting solver timeouts that mask incomplete proofs. The solver happily returns `unsat` in all these cases.

This is not a hypothetical concern. We audited 23 LLM-generated Z3 proof modules on the miniF2F-hard tier (35 AIME and IMO competition problems) and found that **52% are invalid**. The reported solve rate of 65.7% collapses to an actual certified rate of 8.6% once we verify that the encodings are faithful, the derivations are sound, and the proofs are complete. This 57-point

trust gap—between what the solver reports and what is actually proved—is the central finding of this paper.

The problem echoes a broader crisis of trust in LLM evaluation. The PAJAMA study [15] and subsequent "Time to Impeach LLM-as-a-Judge" critiques [16] have demonstrated that LLM-based evaluators systematically over-report quality. Theorem proving was supposed to be different: formal proofs are verifiable by construction. But this guarantee holds only when the proof *certificate* is checked, not merely the solver's return code.

To address this, we propose that SMT-based benchmarks should require **proof certificates**—independently verifiable artifacts that attest to the correctness of both the encoding and the derivation. We implement this using Knuckledragger [11], a Z3 proof assistant developed by Philip Zucker that wraps Z3's proof-producing mode in tamper-proof `Proof` objects, combined with SymPy for algebraic pre-computation. Our certification pipeline (Section 3) achieves 89.3% certified on the full 244-problem miniF2F with Claude Sonnet 4.5, and 70.8% ± 3.1% certified with gpt-5.4-mini across three seeds—numbers that are lower than headline solve rates but *trustworthy*.

Contributions.

1. A systematic audit of LLM-generated Z3 proofs revealing that 52% are invalid, with a taxonomy of seven failure modes (Section 2).
2. A certification pipeline based on Knuckledragger and SymPy that produces independently verifiable proof certificates (Section 3).
3. Experiments on miniF2F, ProofNet, and MATH5 reporting both solve rates and certificate rates, demonstrating a persistent trust gap across models and benchmarks (Section 4).

2 The Audit: What Goes Wrong

We audited 23 Z3 proof modules generated by an LLM pipeline (gpt-5.4-mini, 3 retries) targeting the 35-problem miniF2F-hard tier. Each module was manually inspected by the first author and classified into one of three categories: Valid (faithful encoding + complete derivation), Partial (correct encoding but incomplete derivation or minor gaps), and Invalid (unfaithful encoding or fundamentally unsound reasoning). The results are stark:

Verdict	Count	%
Valid	3	13%
Partial	8	35%
Invalid	12	52%

The reported solve rate for this pipeline was 65.7% (23/35 returned `unsat`). After audit, the actual certified rate is 8.6% (3/35). We identify seven failure modes, summarized in Table 1.

2.1 Illustrative Examples

Vacuous encoding (F1). The most common and most insidious failure. Consider an AIME problem asking for the value of a sum S given certain constraints. A typical invalid script looks like:

Table 1: Taxonomy of failure modes in LLM-generated Z3 proofs (miniF2F-hard, 23 modules audited).

ID	Failure Mode	Count	Description & Example
F1	Vacuous encoding	5	The conclusion is asserted as a premise. E.g., to prove $\sum_{k=1}^{n} k = n(n+1)/2$, the script adds `s.add(S == n*(n+1)/2)` then checks `unsat` of the negation—which is trivially true since the conclusion was assumed.
F2	Assumed derivation	3	Key intermediate lemmas are `assert`ed without proof. The top-level structure is correct, but critical steps are axiomatized rather than derived.
F3	Silent crash / timeout	2	The Z3 script times out or encounters an internal error, but the wrapper script reports `unsat` due to error-handling bugs (e.g., catching `Z3Exception` and returning a default).
F4	Encoding mismatch	4	The Z3 encoding does not faithfully represent the mathematical statement. E.g., an IMO problem about integer divisibility is encoded over reals, losing the integrality constraint that makes the problem non-trivial.
F5	Trivial arithmetic tail	3	The hard mathematical reasoning is hand-waved, and only a trivial final arithmetic check is verified. E.g., "given that the answer is 42, verify $42 = 42$."
F6	Missing quantifiers	2	Universal or existential quantifiers are dropped, converting a $\forall$-statement into a check on a single witness or converting an $\exists$-statement into a vacuous claim.
F7	Partial proof	4	The encoding is faithful and some reasoning is sound, but the proof has gaps—unclosed branches, unhandled edge cases, or lemmas that are stated but not proved.

```python
from z3 import *
s = Solver()
n = Int('n')
S = Int('S')
s.add(n > 0)
s.add(S == n * (n + 1) / 2)  # <-- conclusion as premise!
s.add(Not(S == n * (n + 1) / 2))  # negate "goal"
assert s.check() == unsat  # trivially true
```

Listing 1: Vacuous encoding: the conclusion is asserted as a premise.

This script will always return `unsat` because it asserts P and $\neg P$ simultaneously. The solver is not doing any mathematical work; it is merely detecting a contradiction that the script itself introduced.

Encoding mismatch (F4). An IMO number theory problem requires showing that n^2+1 divides $n!$ for certain n. The LLM encodes this over `Real` sort, computes a real-valued quotient, and checks that it is an integer via `is_int`—but Z3's `is_int` for reals is unreliable for large values and the encoding fails to capture the combinatorial structure of factorials.

Assumed derivation (F2). The proof structure is a chain of lemmas, but the hard lemma is simply asserted:

```python
# "Lemma: for all primes p dividing n, ..."
s.add(ForAll([p], Implies(is_prime(p), divides(p, n))))
# This should be PROVED, not assumed!
```

Listing 2: Assumed derivation: the key lemma is axiomatized.

2.2 Why Standard Z3 Cannot Prevent These Failures

The root cause is that Z3's Python API makes no distinction between *axioms* (trusted assumptions) and *derived facts* (consequences of axioms). Every call to `s.add()` has the same status. A user can add the conclusion as a premise, and Z3 will not object—because from the solver's perspective, it is simply another constraint. This is by design: Z3 is a satisfiability solver, not a proof assistant. It answers "is this set of constraints satisfiable?" without tracking provenance.

This means that a correct Z3 proof requires discipline external to the solver: the human (or LLM) must ensure that only legitimate axioms are asserted, that the encoding is faithful, and that every derivation step is justified. LLMs, trained on pattern matching rather than logical reasoning, systematically fail at this discipline.

3 Certification Methodology

Our certification pipeline addresses the trust gap by requiring every proof to produce a **tamper-proof certificate** that can be independently verified. The pipeline has three components.

3.1 Knuckledragger: Proof-Carrying Z3

Knuckledragger [11] is a Python-based proof assistant built on top of Z3, developed by Philip Zucker. Its key innovation is a `Proof` type that wraps Z3's proof-producing mode and enforces a critical invariant: *you cannot construct a `Proof` object without the kernel verifying the derivation.* Specifically:

- `axiom()` creates a `Proof` for a trusted axiom, explicitly marked as such.
- `lemma(claim, by=[p1, p2, ...])` asks Z3 to derive `claim` from existing proofs `p1, p2, ...` and returns a `Proof` only if the derivation succeeds.
- The `Proof` object is opaque—there is no way to forge one without going through the kernel.

This eliminates failure modes F1 (vacuous encoding) and F2 (assumed derivation) by construction: conclusions cannot be assumed as premises, and every lemma must be derived from prior proofs. Failure mode F3 (silent crashes) is addressed because `lemma()` raises an exception on timeout or failure rather than returning a default.

3.2 SymPy Pre-Computation

Many competition mathematics problems require algebraic manipulation (finding roots of polynomials, simplifying expressions, computing minimal polynomials) that is difficult for Z3's non-linear arithmetic solver. We use SymPy's `minimal_polynomial` and other symbolic routines to pre-compute algebraic facts, which are then fed to Knuckledragger as verified hints.

For example, to certify that $\sqrt{2} + \sqrt{3}$ is a root of $x^4 - 10x^2 + 1$, SymPy computes the minimal polynomial and Knuckledragger verifies the substitution. This division of labor—SymPy for algebraic computation, Knuckledragger for logical verification—is critical for achieving high certificate rates on competition problems.

3.3 Pipeline Architecture (v3)

The full pipeline operates as follows:

1. **Problem parsing**: Extract the mathematical statement and expected answer from the benchmark.
2. **SymPy hints**: Run SymPy to generate algebraic pre-computation hints (minimal polynomials, factorizations, modular arithmetic facts).
3. **Proof generation**: Prompt the LLM to generate a Knuckledragger proof script, providing the problem statement, SymPy hints, and the Knuckledragger API specification.
4. **Multi-attempt**: If the proof fails, retry up to 3 times with error feedback (counterexample-guided repair).
5. **Certificate extraction**: If the proof succeeds, extract the `Proof` object as the certificate.
6. **Certificate upgrade**: Attempt to strengthen partial proofs by adding missing cases or tightening bounds.

A proof is counted as PROVED if the LLM generates a script that returns the correct answer, and as CERTIFIED only if a valid `Proof` object is produced. The gap between these two numbers is the *trust gap*.

4 Experiments

4.1 Setup

We evaluate on three benchmarks:

- **miniF2F** [1]: 244 problems (test split), including 35 hard-tier AIME/IMO problems.
- **ProofNet** [13]: Undergraduate-level mathematical proofs.
- **MATH5**: Level-5 problems from the MATH benchmark [14].

We test four models: GPT-4o, gpt-4o-mini, gpt-5.4-mini, and Claude Sonnet 4.5. All use the v3 pipeline with 3 retries unless otherwise noted. For gpt-5.4-mini, we report multi-seed results (3 seeds) to assess variance.

4.2 The Trust Gap on miniF2F

Table 2 shows the core result: a persistent gap between reported (proved) rates and certified rates across all pipeline versions and models.

Several observations stand out. First, the Z3-only pipeline has an enormous trust gap of 57.1 points—most of what it "proves" is not actually proved. Second, switching to Knuckledragger dramatically reduces but does not eliminate the gap: even with tamper-proof certificates, some proofs that produce correct answers fail certification because the derivation has gaps that the LLM cannot close. Third, the gap narrows with stronger models (Sonnet: 10.7 points) and more retries (see ablation below).

Table 2: Trust gap: proved rate vs. certified rate on miniF2F. "Z3-only" is the pre-audit baseline; KD = Knuckledragger pipeline.

System	Tier	Proved	Certified	Gap
Z3-only (mini)	Hard (35)	65.7%	8.6%	57.1
KD v1 (mini)	Hard (35)	48.6%	37.1%	11.5
KD v3 (mini)	Hard (35)	91.4%	57.1%	34.3
KD v3 (mini)	Full (244)	88.9%	74.2%	14.7
KD v3 (Sonnet)	Full (244)	100%	89.3%	10.7

Table 3: Model comparison on miniF2F (244 problems, KD v3 pipeline, 3 retries). †Multi-seed mean ± std over 3 seeds.

Model	Proved %	Certified %
GPT-4o	76.2	52.0
gpt-4o-mini	68.4	43.9
gpt-5.4-mini†	88.5 ± 0.7	70.8 ± 3.1
Claude Sonnet 4.5	100.0	89.3

4.3 Model Comparison

Table 3 compares four models on the full miniF2F (244 problems) using the KD v3 pipeline.

Claude Sonnet 4.5 achieves the highest certificate rate at 89.3%, solving all 244 problems and certifying 218 of them. The 10.7% uncertified proofs produce correct answers but contain derivation gaps that Knuckledragger rejects. gpt-5.4-mini shows strong performance at 70.8% certified, with low variance across seeds for the proved rate (0.7%) but higher variance for certification (3.1%), suggesting that certification is more sensitive to generation randomness.

4.4 Ablation: Retries Matter

Table 4 shows the effect of multi-attempt retries with counterexample-guided repair on gpt-5.4-mini (miniF2F-hard, 35 problems).

The jump from 2 to 3 retries is disproportionately large, suggesting that the third attempt—informed by two rounds of counterexample feedback—often succeeds where earlier attempts fail. This is consistent with the counterexample-guided repair mechanism: each failed attempt provides diagnostic information that helps the LLM correct encoding errors and derivation gaps.

4.5 Cross-Benchmark Results

Table 5 shows results on ProofNet and MATH5, confirming that the trust gap persists across benchmarks.

ProofNet's undergraduate-level problems are easier to certify than competition problems, with Sonnet achieving 94% certified. MATH5 is harder, with a 10-point gap for Sonnet and a 24-point gap for gpt-5.4-mini. In all cases, the proved rate exceeds the certified rate, reinforcing the need for proof certificates.

Table 4: Ablation: effect of retries on proved/certified rates (gpt-5.4-mini, miniF2F-hard).

Retries	Proved %	Certified %
0	28.6	25.7
1	40.0	31.4
2	48.6	37.1
3 (default)	91.4	57.1

Table 5: Cross-benchmark results (KD v3 pipeline, 3 retries).

Benchmark	Model	Proved %	Certified %
ProofNet	Sonnet 4.5	100	94
	gpt-5.4-mini	88	66
MATH5	Sonnet 4.5	100	90
	gpt-5.4-mini	84	60

4.6 Comparison with Published Systems

Table 6 places our results in context with published systems. We emphasize that **direct comparison is misleading** because Lean-based systems have built-in kernel checking (i.e., their reported rates *are* certificate rates), while our Z3/Knuckledragger certificates serve a different verification role. The comparison is included to contextualize the absolute numbers.

Two comparisons are worth highlighting. First, MiniF2F-Dafny [8] (POPL 2026) is the closest SMT-based competitor at 55.7%; our Sonnet pipeline certifies 89.3%, a 33.6-point improvement, though the verification mechanisms differ. Second, Lean-based systems like Seed-Prover and HILBERT achieve higher absolute numbers, but their certificates are type-checked by the Lean kernel—a property we argue should be the *minimum* standard for all formal verification benchmarks.

5 Related Work

Lean-based theorem proving. The dominant paradigm for LLM-based theorem proving uses Lean 4 as the target language, with the Lean kernel providing built-in proof certification. Seed-Prover [2] combines Monte Carlo Tree Search with a learned value function to achieve near-perfect scores on miniF2F. HILBERT [3] (Apple) uses a similar approach with improved search. Kimina-Prover [4] and Goedel-Prover-V2 [5] explore different training strategies. BFS-Prover [6] applies best-first search. FormalMATH [7] extends formalization to broader mathematical domains. These systems benefit from Lean's kernel guarantee: if the proof type-checks, it is correct. Our work argues that SMT-based systems need an analogous guarantee.

SMT-based approaches. MiniF2F-Dafny [8] (POPL 2026) translates miniF2F problems to Dafny, which compiles to Z3. At 55.7%, it is the strongest published SMT-based result. However, Dafny's verification condition generation provides some certification guarantees that raw Z3 does not. Our Knuckledragger-based approach provides similar guarantees through a different mechanism (proof-carrying code vs. verification condition generation).

Table 6: Comparison with published systems on miniF2F (test split). *Lean kernel provides built-in certification. †Certificate rate; proved rate is higher.

System	Backend	Ref.	Rate (%)
Seed-Prover	Lean 4*	[2]	∼100
HILBERT	Lean 4*	[3]	99.2
Kimina-Prover	Lean 4*	[4]	92.2
Goedel-Prover-V2	Lean 4*	[5]	90.4
AlphaProof	Lean 4*	[9]	—
MiniF2F-Dafny	Dafny/Z3	[8]	55.7
Ours (Sonnet)	KD/Z3		89.3†
Ours (5.4-mini)	KD/Z3		70.8†

Proof integrity in AI mathematics. AlphaProof [9] (DeepMind, Nature 2025) solved IMO problems using AlphaZero-style search with Lean kernel verification—notably choosing to verify proofs through a trusted kernel rather than relying on solver output alone. The Gemini IMO 2025 result [10] similarly relied on formal verification. Our audit demonstrates why this choice matters: without kernel verification, LLM-generated proofs are unreliable.

LLM evaluation skepticism. The PAJAMA study [15] showed that LLM judges systematically overrate response quality, and the "Time to Impeach LLM-as-a-Judge" [16] critique catalyzed a reevaluation of LLM-based evaluation. Our findings extend this skepticism to formal verification: even in a domain where correctness is supposedly checkable, unchecked solver output is unreliable. Recent work on debate-based evaluation [17] similarly highlights the gap between perceived and actual quality in LLM outputs.

Knuckledragger. Knuckledragger [11] was developed by Philip Zucker as a lightweight proof assistant that leverages Z3's proof-producing capabilities while enforcing proof-carrying discipline. The key insight—wrapping Z3 proofs in an opaque `Proof` type—is simple but effective: it transforms Z3 from a satisfiability solver into a proof assistant. We are, to our knowledge, the first to use Knuckledragger for large-scale LLM-based theorem proving.

6 Discussion

All SMT benchmarks should report certificate rates. Our core recommendation is simple: any benchmark that uses SMT solvers (Z3, CVC5, Yices) should report two numbers—the *proved rate* (fraction of problems where the solver returns the expected result) and the *certified rate* (fraction where a tamper-proof certificate is produced). The gap between these numbers is a measure of the benchmark's trustworthiness. For Lean-based benchmarks, these numbers coincide by construction; for SMT-based benchmarks, they typically do not.

The certification tax. Requiring certificates reduces headline numbers. Our best system certifies 89.3% rather than 100% on miniF2F. We view this as a feature, not a bug: the 10.7% gap represents proofs that *actually have gaps*. Reporting 100% would be misleading. The community should reward honest numbers over inflated ones.

Limitations. Our audit was conducted by a single author on a specific set of 23 Z3 modules, and the taxonomy may not cover all failure modes. The Knuckledragger certification pipeline itself relies on Z3's proof-producing mode, which is sound but not complete—some valid proofs may fail to certify due to Z3 limitations rather than proof errors. Our cross-benchmark evaluation (ProofNet, MATH5) uses relatively small problem sets. Finally, miniF2F is saturated: the benchmark's discriminative power is limited when top systems approach 100%, and our contribution is better understood as a methodology (proof certificates) than a leaderboard result.

Future work. We plan to extend the certification pipeline to larger benchmarks (FormalMATH [7], SciFact, QuAD), investigate progressive RAG for proof retrieval, and explore whether debate-based self-verification [17] can improve certificate rates without requiring a trusted kernel.

Acknowledgments

We thank Philip Zucker for developing Knuckledragger and for helpful discussions about proof-carrying code in SMT solvers.

References

[1] K. Zheng, J. M. Han, and S. Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

[2] Seed-Prover Team. Seed-Prover: Large language models as formal theorem provers via MCTS. *arXiv preprint arXiv:2507.23726*, 2025.

[3] Apple ML Research. HILBERT: A large language model approach to formal theorem proving. *arXiv preprint arXiv:2509.22819*, 2025.

[4] Kimina Team. Kimina-Prover: Scaling formal theorem proving with reinforcement learning. *arXiv preprint*, 2025.

[5] Goedel Team. Goedel-Prover-V2: Improved neural theorem proving via synthetic data and curriculum learning. *arXiv preprint*, 2025.

[6] BFS-Prover Team. BFS-Prover: Best-first search for formal theorem proving. *arXiv preprint*, 2025.

[7] FormalMATH Team. FormalMATH: Scaling formal mathematics with language models. *arXiv preprint*, 2025.

[8] Y. Sun, Y. Wu, and X. Si. Formal verification of LLM-generated mathematical proofs via Dafny. In *Proceedings of the ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2026.

[9] DeepMind AlphaProof Team. AlphaProof: AI achieves silver-medal level performance at the International Mathematical Olympiad. *Nature*, 2025.

[10] Google DeepMind. Gemini solves IMO problems with formal verification. *Google DeepMind Blog*, 2025.

[11] P. Zucker. Knuckledragger: A Python proof assistant built on Z3. https://github.com/philzook58/knuckledragger, 2026.

[12] P. Zucker. The state of Knuckledragger III. *Blog post*, March 2026.

[13] Z. Azerbayev, B. Piotrowski, and J. Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint*, 2023.

[14] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

[15] PAJAMA Authors. PAJAMA: Systematic evaluation of LLM-as-a-judge. *arXiv preprint*, 2025.

[16] M. Chen and colleagues. Time to impeach LLM-as-a-judge: Systematic biases in automated evaluation. *arXiv preprint*, 2025.

[17] N. Kenton, T. Everitt, and J. Leike. Debate does not (yet) help humans identify deceptive AI. In *International Conference on Learning Representations (ICLR)*, 2025.